

```
❶ >>> Path.cwd()
WindowsPath('C:/Users/Al/AppData/Local/Programs/Python/Python38')
❷ >>> os.chdir('C:\\Windows\\System32')
>>> Path.cwd()
WindowsPath('C:/Windows/System32')
```

Here, the `cwd` was set to `C:\Users\Al\AppData\Local\Programs\Python\Python38❶`, so the filename `project.docx` would refer to `C:\Users\Al\AppData\Local\Programs\Python\Python38\project.docx`. When we change the `cwd` to `C:\Windows\System32❷`, the filename `project.docx` would refer to `C:\Windows\System32\project.docx`.

Python displays an error if you try to change to a directory that doesn't exist:

```
>>> os.chdir('C:/ThisFolderDoesNotExist')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [WinError 2] The system cannot find the file specified:
'C:/ThisFolderDoesNotExist'
```

The `os.getcwd()` function in the `os` module is a former way of getting the `cwd` as a string.

Absolute vs. Relative Paths

There are two ways to specify a file path:

- An absolute path, which always begins with the root folder
- A relative path, which is relative to the program's `cwd`

There are also the *dot* (`.`) and *dot-dot* (`..`) folders. These are not real folders but special names that you can use in a path. A single period (`.`) for a folder name is shorthand for “this directory.” Two periods (`..`) means “the parent folder.”

[Figure 2-2](#) shows an example of some folders and files. When the `cwd` is set to `C:\bacon`, the relative paths for the other folders and files are set as they are in the figure.